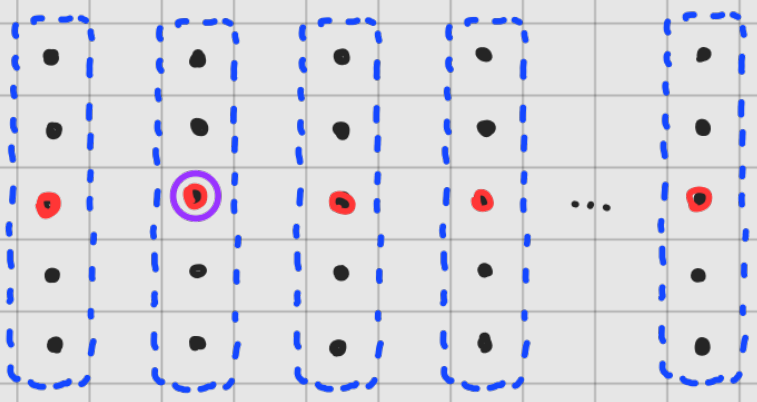
Tutorato AsD ~ L8 — 30/04/2026

Esercizio 1. Nell'algoritmo SELECT gli elementi di input sono suddivisi in gruppi di 5. Dimostrate che l'algoritmo opera in tempo lineare anche se gli elementi sono suddivisi in gruppi di 7.

```
RANDOMIZED-SELECT(A, p, r, i)
1  if p == r
2      return A[p]        // 1 ≤ i ≤ r − p + 1 when p == r means that i = 1
3  q = RANDOMIZED-PARTITION(A, p, r)
4  k = q − p + 1
5  if i == k
6      return A[q]        // the pivot value is the answer
7  elseif i < k
8      return RANDOMIZED-SELECT(A, p, q − 1, i)
9  else return RANDOMIZED-SELECT(A, q + 1, r, i − k)
```

```
SELECT(A, p, r, i)
 1  while (r − p + 1) mod 5 ≠ 0
 2      for j = p + 1 to r                  // put the minimum into A[p]
 3          if A[p] > A[j]
 4              exchange A[p] with A[j]
 5      // If we want the minimum of A[p : r], we're done.
 6      if i == 1
 7          return A[p]
 8      // Otherwise, we want the (i − 1)st element of A[p + 1 : r].
 9      p = p + 1
10      i = i − 1
11  g = (r − p + 1)/5                       // number of 5-element groups
12  for j = p to p + g − 1                  // sort each group
13      sort ⟨A[j], A[j + g], A[j + 2g], A[j + 3g], A[j + 4g]⟩ in place
14  // All group medians now lie in the middle fifth of A[p : r].
15  // Find the pivot x recursively as the median of the group medians.
16  x = SELECT(A, p + 2g, p + 3g − 1, ⌈g/2⌉)
17  q = PARTITION-AROUND(A, p, r, x)    // partition around the pivot
18  // The rest is just like lines 3–9 of RANDOMIZED-SELECT.
19  k = q − p + 1
20  if i == k
21      return A[q]                         // the pivot value is the answer
22  elseif i < k
23      return SELECT(A, p, q − 1, i)
24  else return SELECT(A, q + 1, r, i − k)
```

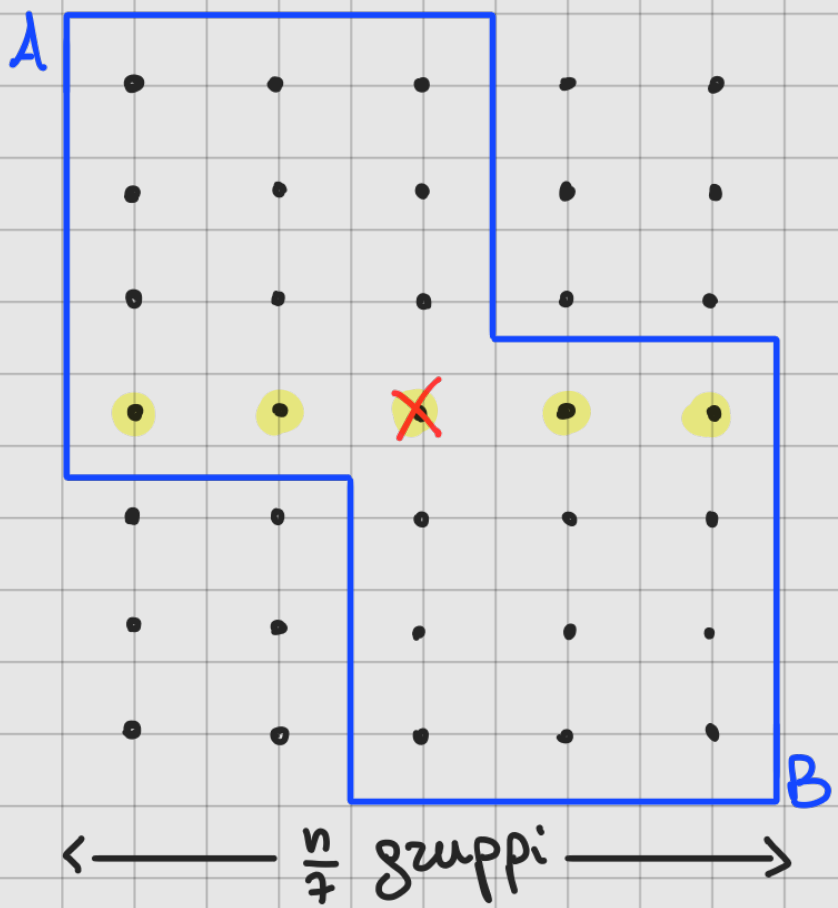


- Divido gli n elementi in gruppi da 5.
- Trovo la mediana di ciascuno
- Trovo x la mediana delle mediane
- Uso x come pivot per PARTITION

$\rightsquigarrow$ Il numero di elementi $\leq x$ è almeno $3 \cdot (n/5) \cdot (1/2) = \frac{3n}{10}$

↳ Al più $\frac{7n}{10}$

$$T(n) = T(n/5) + T(7n/10) + \Theta(n)$$



$\frac{n}{7}$ gruppi $\rightarrow \frac{n}{7}$ mediane

A contiene el. $\leq x$

B contiene el. $\geq x$

$|A| \geq 4 \cdot \frac{n}{7} \cdot \frac{1}{2} = \frac{2}{7} n$

↳ el $\geq x$ sono al più $\frac{5}{7} n$

Discorso analogo per B

$\rightsquigarrow T(n) = T(\frac{1}{7}n) + T(\frac{5}{7}n) + \Theta(n)$ $\quad \frac{1}{7} + \frac{5}{7} < 1 \Rightarrow$ Oss. Lez 20

Esercizio 2. Spiegate come QUICKSORT possa essere eseguito in $O(n \log n)$ nel *caso peggiore*, supponendo che tutti gli elementi siano distinti.

Sia A array di n elementi distinti da ordinare.

In ogni chiamata a QUICKSORT, invoco SELECT per trovare mediana inferiore di sottoarray in input e suo indice.

Poi chiamo PARTITION con Pivot la mediana inf. $\lfloor (n+1)/2 \rfloor$

In questo modo ho sempre partizione bilanciata.

$$T(n) = \underbrace{2T\left(\frac{n}{2}\right)}_{\text{RECURSIVE CALL a QUICKSORT su SUB-ARR BILANCIATI}} + \underbrace{O(n)}_{\text{SELECT}} + \underbrace{O(n)}_{\text{PARTITION}} = O(n \log n)$$

Esercizio 3. Supponete di avere una subroutine black-box che trova la mediana in tempo lineare nel caso peggiore. Create un semplice algoritmo che risolve in tempo lineare il problema della selezione per un'arbitraria statistica d'ordine.

Sia MEDIAN(A, p, q) black-box che trova indice mediana di A[p..q] in $\Theta(n)$ nel caso peggiore.

→ MODIFICO RAND-SELECT

TROVA i-esima stat. d'ordine di A[p..q]

NEW-SELECT(A, p, q, i):

```
if p=q return A[p]
s <- MEDIAN(A, p, q)
swap A[p] & A[s]
r <- PARTITION(A, p, q)      partiziono attorno a MEDIANA
k <- r-p+1                   rango di A[r]
if i = k return A[r]
if i < k
    return NEW-SELECT(A, p, r-1, i)
else
    return NEW-SELECT(A, r+1, q, i-k)
```



$$T(n) = T(n/2) + \Theta(n) = T(n)$$

Esercizio 4. Descrivete un algoritmo con tempo $O(n)$ che, dato un insieme S di n numeri distinti ed un intero positivo $k \leq n$, trova i k numeri in S più vicini alla mediana di S.

Sia $S[1..n]$ array.

1. Trovo la mediana m di S con SELECT
2. Sia $S'[1..n]$ nuovo array

   con $S'[i] = |S[i] - m| \quad \forall i \in \{1, \dots, n\}$
3. Trovo k-esima statistica d'ordine m_k di S' con SELECT
4. Scan di S per trovare tutti gli elem. $S[i]$ t.c.

   $|S[i] - m| \leq m_k$

Ciascuno dei 4 passi costa $O(n)$

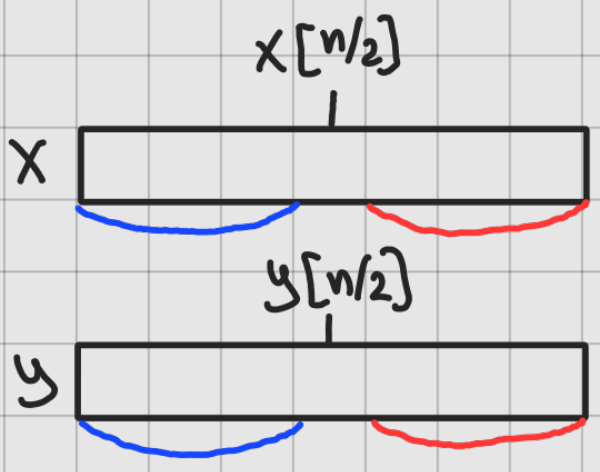
↳ $T(n) = O(n)$

Esercizio 5. Siano $X[1..n]$ e $Y[1..n]$ due array ordinati. Descrivete un algoritmo con tempo $O(\log n)$ per trovare la mediana di tutti i $2n$ elementi degli array X e Y.

(MEDIANA inf)

Se faccio MERGE e seleziono la mediana → $O(n)$ ✗

↳ Uso l'idea del BINARY-SEARCH



2 CASI

1) $X[n/2] < y[n/2]$

↳ Cerco la MEDIANA in

$X[n/2+1..n]$ & $y[1..n/2]$

2) $X[n/2] > y[n/2]$

↳ in $X[1..n/2]$ & $y[n/2+1..n]$

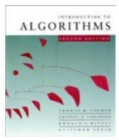
```
MEDIAN(X, y, n)
  if (n=1) return min{X[1], y[1]}
  if X[n/2] < y[n/2]
     return MEDIAN(X[n/2+1..n], y[1..n/2], n/2)
  return MEDIAN(X[1..n/2], y[n/2+1..n], n/2)
```

$$T(n) = T(n/2) + \Theta(1) = T(\log n)$$

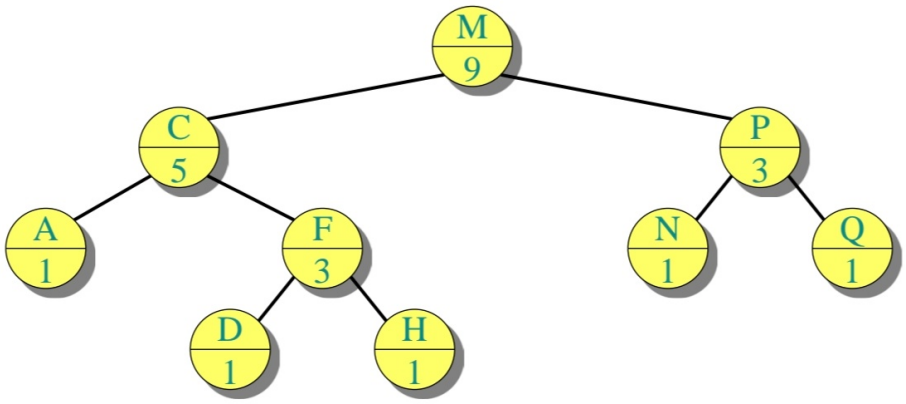
Esercizio 6. Dati un elemento x in un albero per statistiche d'ordine di n nodi e $i \in \mathbb{N}$, come può essere determinato nel tempo $O(\log n)$ l'i-esimo successore di x in una visita in-order dell'albero?

OS-SELECT(i, S): returns the ith smallest element in the dynamic set S.

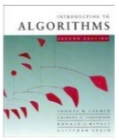
OS-RANK(x, S): returns the rank of $x \in S$ in the sorted order of S's elements.



Example of an OS-tree



$$size[x] = size[left[x]] + size[right[x]] + 1$$



Selection

Implementation trick: Use a ***sentinel*** (dummy record) for NIL such that $size[\text{NIL}] = 0$.

OS-SELECT(x, i) ▷ ith smallest element in the subtree rooted at x

```
k ← size[left[x]] + 1    ▷ k = rank(x)
if i = k then return x
if i < k
    then return OS-SELECT(left[x], i)
    else return OS-SELECT(right[x], i − k)
```

OS-RANK(T, x)

```
r ← size[left[x]] + 1
y ← x
while y ≠ root[T]
    do if y = right[p[y]]
          then r ← r + size[left[p[y]]] + 1
       y ← p[y]
return r
```

L'i-esimo successore di x in una visita in-order è per definizione l'elem. con rango rank(x)+i in T.

OS-SELECT(T, OS-RANK(T, x) + i)

Sia OS-SELECT che OS-RANK hanno exec-time $O(\log n)$

↳ $T(n) = O(\log n) + O(\log n) = O(\log n)$

Esercizio 7. Una vecchia conoscenza. Spiegate come utilizzare un albero per statistiche d'ordine per contare in $O(n \log n)$ il numero di inversioni in un array di dimensione n.

Ricordiamo che una **inversione** in $A[1..n]$ è una coppia (i,j) con $i<j$ e $A[i]>A[j]$

Idea: Processiamo l'array da destra a sinistra mantenendo un OS-tree T degli elementi già visitati.

Quando si processa $A[i]$, il numero di inversioni in cui i è il membro sx della coppia è pari al numero di elementi in $\{A[i+1], \dots, A[n]\}$ strettamente minori di $A[i]$.

Questo è esattamente OS-RANK$(T, A[i]) - 1$ dopo aver inserito $A[i]$ in T.

COUNT-INVERSIONS(A, n): $\quad T(n) = O(n \log n)$

```
T ← empty OS-tree
inv ← 0
for i = n downto 1
    INSERT(T, A[i])
*   inv ← inv + OS-RANK(T, A[i]) - 1
return inv
```

$O(\log n)$ dentro $O(n \log n)$

* CORRETTEZZA: Qui T contiene $\{A[i], A[i+1], \dots, A[n]\}$.

Si ha OS-RANK$(T, A[i]) - 1 = |\{j > i : A[i] > A[j]\}|$, cioè esattamente le inversioni (i, j) con i membro sx.

Esercizio 8. È possibile gestire le altezze nere dei nodi di un albero rosso-nero come attributi dei nodi dell'albero senza influire sulle prestazioni asintotiche delle altre operazioni? E riguardo alla possibilità di gestire le profondità dei nodi (i.e., le distanze dalla radice)?

Altezze Nere: Sí, é possibile!

L'altezza nera $bh(x)$ conta i nodi su qualunque cammino da x (escluso) a una foglia NIL. É un attributo **bottom-up**: dipende solo dai figli di x.

$$bh(NIL) = 0, \quad bh(x) = \begin{cases} bh(x.left) + 1, & \text{se } x.left \text{ é BLACK} \\ bh(x.left), & \text{altrimenti} \end{cases}$$

Le proprietá RB garantiscono che il conto é identico usando $x.right$. L'attributo puó essere ricalcolato in $O(1)$ da x e dagli attributi dei figli, senza risalire verso la radice né scendere verso le foglie: soddisfa esattamente le ipotesi del THM dell'aumento di un RB-tree. INSERT e DELETE eseguono $O(\log n)$ rotazioni/ricolorazioni nel fixup; ciascuna richiede $O(1)$ per aggiornare bh dei nodi coinvolti → nessun impatto asintotico.

Profonditá: No, non é possibile

La profonditá é un attributo **top-down**:

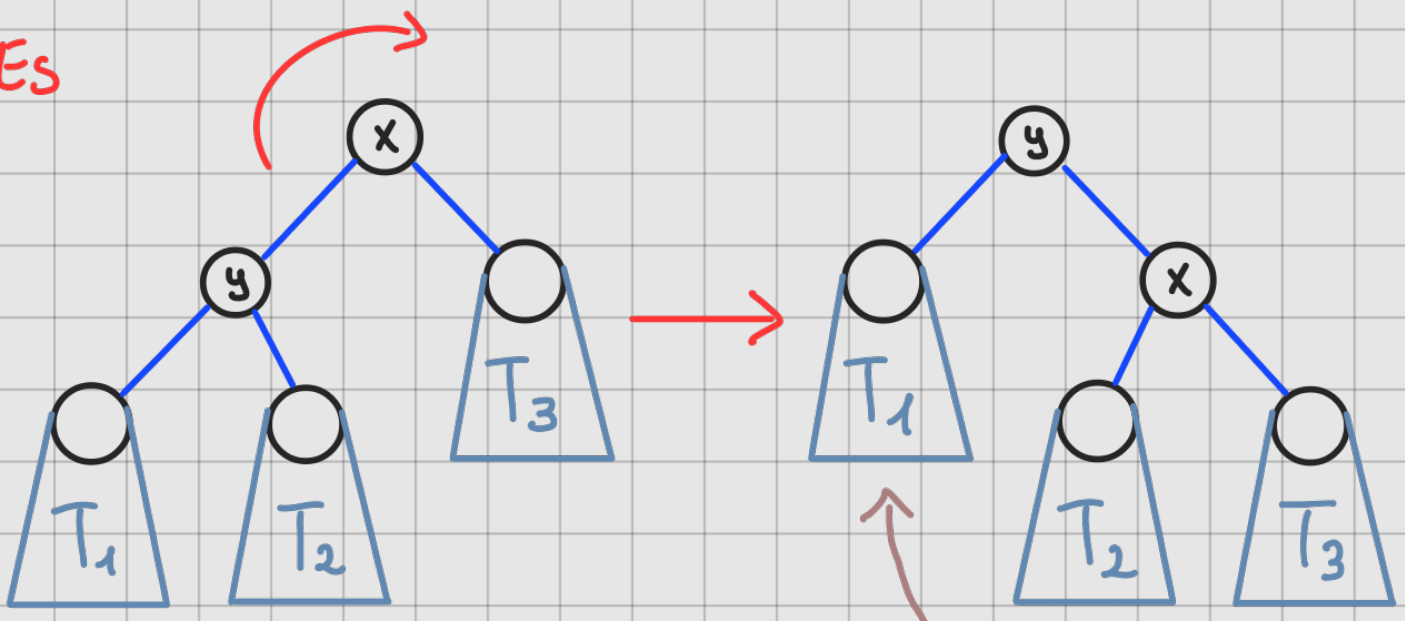
$$depth(x) = depth(x.parent) + 1$$

Dipende dal genitore, non dai figli
↳ il THM non é applicabile

Infatti, una rotazione potrebbe modificare la profonditá

di $\Omega(n)$ nodi → Costo di INSERT/DELETE non più $O(\log n)$

Es



profondità nodi in T_1 e T_3 cambiano

Esercizio 9. Spiegate come gestire un insieme dinamico Q di numeri che supporta, oltre a INSERT, DELETE e SEARCH, anche l'operazione MIN-GAP; questa operazione fornisce il valore della differenza tra i due numeri più vicini in Q. Per esempio, se $Q = \{1, 5, 9, 15, 18, 22\}$, allora MIN-GAP($Q$) restituisce $18 - 15 = 3$.

Usiamo una struttura RB-tree aumentata

x.min_val	chiave minima nel sottoalbero radicato in x
x.max_val	chiave massima nel sottoalbero radicato in x
x.min_gap	gap minimo tra chiavi consecutive (in-order) nel sottoalbero radicato in x

$x.min_val = \min\{x.key, x.left.min_val, x.right.min_val\}$

$x.max_val = \min\{x.key, x.left.max_val, x.right.max_val\}$

$x.min_gap = \min\{x.left.min_gap, x.right.min_gap,$
$x.key - x.left.max_val, x.right.min_val - x.key\}$

Convenzione: $NIL.min_val = +\infty$; $NIL.max_val = -\infty$;
$NIL.min_gap = +\infty$

Definiamo una funzione UPDATE che verrà chiamata BOTTOM-UP sui nodi coinvolti in ogni rotazione e ricolorazione durante RB-INSERT-FIXUP e RB-DELETE-FIXUP

UPDATE(x):

$l \leftarrow x.left$; $r \leftarrow x.right$

$x.min_val \leftarrow (l \neq NIL)\ ?\ l.min_val : x.key$

$x.max_val \leftarrow (r \neq NIL)\ ?\ r.max_val : x.key$

$x.min_gap \leftarrow +\infty$

if $l \neq NIL$:

$x.min_gap \leftarrow \min\{x.min_gap, l.min_gap\}$

$x.min_gap \leftarrow \min\{x.min_gap, x.key - l.max_val\}$

if $r \neq NIL$:

$x.min_gap \leftarrow \min\{x.min_gap, r.min_gap\}$

$x.min_gap \leftarrow \min\{x.min_gap, r.min_val - x.key\}$

UPDATE ha costo $O(1)$. Viene chiamata durante il fixup per un costo totale di $O(\log n)$

MIN-GAP(T):

return T.root.min_gap $O(1)$

Esercizio 10 (1D Range Search). Consideriamo un caso semplice di query in un database. Vogliamo aumentare gli alberi rosso-neri con un'operazione RB-ENUMERATE(x, a, b) che genera in output tutte le chiavi k tali che $a \leq k \leq b$ in un albero rosso-nero con radice x. Spiegate come implementare RB-ENUMERATE in $O(m + \log n)$, dove m è il numero di chiavi in output e n è il numero di nodi interni dell'albero. Non occorre aggiungere nuovi attributi all'albero rosso-nero.

```
RB-ENUMERATE(x, a, b):

1  if x = NIL:
2      return

3  if a ≤ x.key:
4      RB-ENUMERATE(x.left, a, b)

5  if a ≤ x.key ≤ b:
6      output x.key

7  if x.key ≤ b:
8      RB-ENUMERATE(x.right, a, b)
```

I nodi visitati si dividono in 2 categorie:

1. Nodi in range (chiave in $[a,b]$):
   ↳ Sono esattamente m, tutti prodotti in output $O(m)$

2. Nodi boundary (chiave fuori da $[a,b]$):

   - Se x.key < a: la riga 2 è falsa, la riga 4 è falsa. L'algoritmo ricorre solo a dx (righe 6-7). Questi nodi $O(\log n)$ formano un cammino verso il più piccolo elem. $\geq a$
   - Se x.key > b è analogo. $O(\log n)$

$$T = O(m + \log n)$$